

Rural Detention

Table of contents

1	RUCA Codes	2
2	ACS Data	4
2.1	Cost Burden	4
2.2	Poverty Rate	6
2.3	Housing Quality	7
2.4	Education	8
2.5	Demographics	9
2.6	Household Vars	10
3	Data Analysis	10
3.1	Descriptive Statistics	12
3.2	Pearson Correlation Matrix	15
3.3	Variable Inflation Factor	17
3.4	Multinomial Logistic Regression	18
4	Spatial Analysis	24

```
import pandas as pd
import numpy as np
from dotenv import load_dotenv
import pytidycensus as tc
from os import getenv
from concurrent.futures import ThreadPoolExecutor
from IPython.display import HTML, display
import matplotlib.pyplot as plt
import statsmodels.api as sm
from sklearn.metrics import confusion_matrix, accuracy_score,
classification_report
from statsmodels.stats.outliers_influence import variance_inflation_factor
from pygrids import tracts

import seaborn as sns
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
import matplotlib.pyplot as plt
from utils import collect_ruca_data
```

```
load_dotenv()

API_KEY: str = getenv("CENSUS_KEY")

tc.set_census_api_key(API_KEY)
```

Census API key has been set for this session.

1 RUCA Codes

Code	Classification description
1	Metropolitan core: primary flow is within an urban area of 50,000 or more people (metro UA)
2	Metropolitan high commuting: primary flow is 30 percent or more to a metro UA
3	Metropolitan low commuting: primary flow is 10 percent to 30 percent to a metro UA
4	Micropolitan core: primary flow is within an urban area of 10,000 to 49,999 people (micro UA)
5	Micropolitan high commuting: primary flow is 30 percent or more to a micro UA
6	Micropolitan low commuting: primary flow is 10 percent to 30 percent to a micro UA
7	Small town core: primary flow is within an urban area of 9,999 or fewer people (small town UA)
8	Small town high commuting: primary flow is 30 percent or more to a small town UA
9	Small town low commuting: primary flow is 10 percent to 30 percent to a small town UA
10	Rural area: primary flow is to a tract outside an UA

Code	Classification description
99	Not coded: census tract is entirely in coastal or inland water bodies, with zero population and zero land area

See the RUCA Codes documentation for detailed info on data and methods used for creating RUCA Codes.

```
if "ruca" not in locals():
    ruca = collect_ruca_data()
ruca.head()
```

	TractFIPS	TractName	County-FIPS	County-Name	StateFIPS	StateName	RUCA
0	01001020100	Census Tract 201	01001	Autauga County	01	Alabama	1
1	01001020200	Census Tract 202	01001	Autauga County	01	Alabama	1
2	01001020300	Census Tract 203	01001	Autauga County	01	Alabama	1
3	01001020400	Census Tract 204	01001	Autauga County	01	Alabama	1
4	01001020501	Census Tract 205.01	01001	Autauga County	01	Alabama	1

```
STATES = list(ruca["StateFIPS"].unique()) # List of states to pass to
pytidycensus
YEAR: int = 2024
```

```
ruca_counts = {
    "Count": ruca["RUCA"].value_counts().sort_index(),
    "Proportion": round(
        ruca["RUCA"].value_counts(normalize=True).sort_index() * 100, 2
    ),
}
pd.DataFrame(ruca_counts)
```

	Count	Proportion
RUCA		
1	57544	68.69
2	8481	10.12
3	1149	1.37
4	4630	5.53
5	2224	2.65
6	693	0.83
7	1475	1.76
8	607	0.72
9	336	0.40
10	6343	7.57
99	294	0.35

2 ACS Data

2.1 Cost Burden

Collects variables for housing cost burden (renters and owners spending 30% or more of household income on housing) to measure overall cost burden.

```
rent_burden_vars = [
    "B25070_001E", # Total (Renter)
    "B25070_007E", # Gross rent: 30.0 to 34.9 percent
    "B25070_008E", # Gross rent: 35.0 to 39.9 percent
    "B25070_009E", # Gross rent: 40.0 to 49.9 percent
    "B25070_010E", # Gross rent: 50.0 percent or more
]

owner_burden_vars = [
    "B25091_001E", # Total (Owner)
    "B25091_008E", # Housing units with a mortgage: 30.0 to 34.9 percent
    "B25091_009E", # Housing units with a mortgage: 35.0 to 39.9 percent
    "B25091_010E", # Housing units with a mortgage: 40.0 to 49.9 percent
    "B25091_011E", # Housing units with a mortgage: 50.0 percent or more
    "B25091_019E", # Housing units without a mortgage: 30.0 to 34.9 percent
    "B25091_020E", # Housing units without a mortgage: 35.0 to 39.9 percent
    "B25091_021E", # Housing units without a mortgage: 40.0 to 49.9 percent
    "B25091_022E", # Housing units without a mortgage: 50.0 percent or more
]
```

```

]

cost_burden_vars = rent_burden_vars + owner_burden_vars

var_table_rent = tc.get_table_variables("B25070", 2024, "acs", "acs5")
var_table_owner = tc.get_table_variables("B25091", 2024, "acs", "acs5")
var_table = pd.concat([var_table_rent, var_table_owner])
var_table = var_table.loc[var_table["name"].isin(cost_burden_vars)]
var_table["label"] = var_table["label"].str.replace("Estimate", "")
var_table["label"] = var_table["label"].str.replace("Total", "")
var_table["label"] = var_table["label"].str.replace("!!", "")

# var_table

```

```

Loaded cached variables for 2024 acs acs5
Loaded cached variables for 2024 acs acs5

```

```

if "cost_burden" not in locals():
    cost_burden = tc.get_acs(
        geography="tract", variables=cost_burden_vars, state=STATES, year=YEAR
    )

    # Filter out tracts with zero total units (both renter and owner units
    are zero)
    total_units = cost_burden["B25070_001E"] + cost_burden["B25091_001E"]
    # cost_burden = cost_burden.loc[total_units != 0].dropna(axis=0)

    # Re-evaluate total_units after filtering
    total_units = cost_burden["B25070_001E"] + cost_burden["B25091_001E"]

    # Compute combined cost burden
    burdened_renters = cost_burden[rent_burden_vars[1:]].sum(axis=1)
    burdened_owners = cost_burden[owner_burden_vars[1:]].sum(axis=1)
    cost_burden["CostBurden"] = (burdened_renters + burdened_owners) /
    total_units

    cost_burden = cost_burden[["GE0ID", "CostBurden"]]

cost_burden.head()

```

	GEOID	CostBurden
0	01001020100	0.168478
1	01001020200	0.163851
2	01001020300	0.235628
3	01001020400	0.142857
4	01001020501	0.189303

2.2 Poverty Rate

Calculates poverty rate based on the number of people with income below the poverty level in 12 months and divides it by the population.

```
poverty_vars = [
    "B17001_001E", # Total
    "B17001_002E", # Income in the past 12 months below poverty level
]

var_table = tc.get_table_variables("B17001", 2024, "acs", "acs5")
var_table = var_table.loc[var_table["name"].isin(poverty_vars)]
var_table["label"] = var_table["label"].str.replace("Estimate", "")
var_table["label"] = var_table["label"].str.replace("Total", "")
var_table["label"] = var_table["label"].str.replace("!!", "")

# var_table
```

Loaded cached variables for 2024 acs acs5

```
if "poverty" not in locals():
    poverty = tc.get_acs(
        geography="tract", variables=poverty_vars, state=STATES, year=YEAR
    )

    poverty["PovertyRate"] = poverty[poverty_vars[1]] / poverty[poverty_vars[0]]
    poverty = poverty[["GEOID", "PovertyRate"]]

poverty.head()
```

	GEOID	PovertyRate
0	01001020100	0.174060
1	01001020200	0.095051

	GEOID	PovertyRate
2	01001020300	0.071387
3	01001020400	0.153495
4	01001020501	0.035104

2.3 Housing Quality

Collects variables for substandard plumbing and kitchen facilities to measure overall housing quality.

```
housing_quality_vars = [
    "B25047_001E", # Total units (Plumbing)
    "B25047_003E", # Lacking complete plumbing facilities
    "B25051_001E", # Total units (Kitchen)
    "B25051_003E", # Lacking complete kitchen facilities
]

# Fetch metadata for both tables and concatenate them
var_table_plumbing = tc.get_table_variables("B25047", 2024, "acs", "acs5")
var_table_kitchen = tc.get_table_variables("B25051", 2024, "acs", "acs5")
var_table = pd.concat([var_table_plumbing, var_table_kitchen])
var_table = var_table.loc[var_table["name"].isin(housing_quality_vars)]
var_table["label"] = var_table["label"].str.replace("Estimate", "")
var_table["label"] = var_table["label"].str.replace("Total", "")
var_table["label"] = var_table["label"].str.replace("!!", "")

# var_table
```

```
Loaded cached variables for 2024 acs acs5
Loaded cached variables for 2024 acs acs5
```

```
if "housing_quality" not in locals():
    housing_quality = tc.get_acs(
        geography="tract", variables=housing_quality_vars, state=STATES, year=YEAR
    )

    # Filter out tracts with zero total units and compute index
    # housing_quality
    = housing_quality.loc[housing_quality[housing_quality_vars[0]] !=
0].dropna(axis=0)
    housing_quality["HousingQuality"] = (
```

```

        housing_quality["B25047_003E"] + housing_quality["B25051_003E"]
    ) / housing_quality["B25047_001E"]

    housing_quality = housing_quality[["GEOID", "HousingQuality"]]

housing_quality.head()

```

	GEOID	HousingQuality
0	01001020100	0.007916
1	01001020200	0.100287
2	01001020300	0.092125
3	01001020400	0.008419
4	01001020501	0.000000

2.4 Education

Calculates percentage of the population that does not hold a high school diploma

```

education_vars = [
    "B15003_001E", # Total (Population 25 years and over)
] + [f"B15003_{num:03d}E" for num in range(2, 17)]

var_table = tc.get_table_variables("B15003", 2024, "acs", "acs5")
var_table = var_table.loc[var_table["name"].isin(education_vars)]
var_table["label"] = var_table["label"].str.replace("Estimate", "")
var_table["label"] = var_table["label"].str.replace("Total", "")
var_table["label"] = var_table["label"].str.replace("!!", "")

# var_table

```

Loaded cached variables for 2024 acs acs5

```

if "education" not in locals():
    education = tc.get_acs(
        geography="tract", variables=education_vars, state=STATES, year=YEAR
    )

# education = education.loc[education[education_vars[0]] != 0].dropna(axis=0)
education["LessThanHS"] = (
    education[education_vars[1:]].sum(axis=1) / education[education_vars[0]]
)

```

```
)
education = education[["GEOID", "LessThanHS"]]

education.head()
```

	GEOID	LessThanHS
0	01001020100	0.094197
1	01001020200	0.097473
2	01001020300	0.130983
3	01001020400	0.053578
4	01001020501	0.031766

2.5 Demographics

```
demographics_vars = [
    "B03002_001E", # Total
    "B03002_003E", # Not Hispanic or Latino: White alone
]

var_table = tc.get_table_variables("B03002", 2024, "acs", "acs5")
var_table = var_table.loc[var_table["name"].isin(demographics_vars)]
var_table["label"] = var_table["label"].str.replace("Estimate", "")
var_table["label"] = var_table["label"].str.replace("Total", "")
var_table["label"] = var_table["label"].str.replace("!!", "")

# var_table
```

Loaded cached variables for 2024 acs acs5

```
if "demographics" not in locals():
    demographics = tc.get_acs(
        geography="tract", variables=demographics_vars, state=STATES, year=YEAR
    )

    # demographics = demographics.loc[demographics[demographics_vars[0]] != 0].dropna(axis=0)
    demographics["PercentMinority"] = 1 - (
        demographics[demographics_vars[1]] / demographics[demographics_vars[0]]
    )
    demographics = demographics[["GEOID", "PercentMinority"]]
```

```
demographics.head()
```

	GEOID	PercentMinority
0	01001020100	0.334643
1	01001020200	0.676971
2	01001020300	0.358540
3	01001020400	0.119048
4	01001020501	0.215122

2.6 Household Vars

```
misc_vars = {
    "Gini": "B19083_001E",
}

if "misc" not in locals():
    misc = tc.get_acs(
        geography="tract",
        variables=misc_vars,
        state=STATES,
        year=YEAR,
    )

misc = misc[["GEOID", "Gini"]]
misc.head()
```

	GEOID	Gini
0	01001020100	0.4238
1	01001020200	0.4100
2	01001020300	0.3519
3	01001020400	0.5880
4	01001020501	0.3954

3 Data Analysis

Merge all individual ACS socioeconomic indicators and combines them with the cleaned RUCA dataset by Census Tract GEOID.

```

if "DATA" not in locals():
    # List of collected ACS dataframes
    collected_dfs = [
        poverty,
        cost_burden,
        housing_quality,
        education,
        demographics,
        misc,
    ]

    acs_merged = collected_dfs[0]
    for df in collected_dfs[1:]:
        acs_merged = pd.merge(acs_merged, df, on="GE0ID", how="outer")

    print(f"Total ACS tracts merged: {len(acs_merged)}")

    DATA = pd.merge(
        ruca, acs_merged, left_on="TractFIPS", right_on="GE0ID", how="inner"
    )

    print(f"Final tract count: {len(DATA)}")

DATA.head()

```

	Tract- FIPS	Tract- Name	Coun- FIPS	Coun- Name	State- FIPS	State- Name	RUCAGE0ID	Pover- tyRate	Cost- Burden	Less Than HS Qual- ity	Per- cent- Minor- ity	Gini					
0	01001020100	01001 sus Tract 201	01001	Autauga County	01	Alabama	01001020100	17.40	60.16	84.78	00.79	16.09	41.97	33.46	46.42	38	
1	01001020200	01001 sus Tract 202	01001	Autauga County	01	Alabama	01001020200	25.50	51.16	38.51	11.00	28.70	9.74	73.67	69.70	14.10	0
2	01001020300	01001 sus Tract 203	01001	Autauga County	01	Alabama	01001020300	37.87	23.56	28.09	21.25	13.09	8.33	58.54	40.35	19	0
3	01001020400	01001 sus Tract 204	01001	Autauga County	01	Alabama	01001020400	34.95	14.28	37.00	08.41	19.05	35.78	11.19	46.85	88.0	0

Tract- FIPS	Tract- Name	Coun- ty	Coun- ty	State- Name	State- Name	RUCAGEOID	Pover- tyRate	Cost- Bur- den	Hous- Less- Than- HS ingQual- ity	Per- cent- Minor- ity	Gini
01001025014	205.01	01001	Autauga	Alabama	01001025014	01001025014	0.135813	0.312121	0.044292	0.109586	0.423781

```
df = DATA.replace([np.inf, -np.inf], np.nan).dropna()
```

```
len(DATA), len(df)
```

```
(82879, 81802)
```

3.1 Descriptive Statistics

```
df.describe()
```

	RUCA	Pover- tyRate	CostBur- den	Hous- Less- Than- HS ingQual- ity	Percent- Minority	Gini
count	81802.000000	81802.000000	81802.000000	81802.000000	81802.000000	81802.000000
mean	2.314088	0.135813	0.312121	0.044292	0.109586	0.423781
std	2.628321	0.108560	0.125279	0.069683	0.096994	0.067424
min	1.000000	0.000000	0.000000	0.000000	0.000000	0.000100
25%	1.000000	0.058684	0.217634	0.000000	0.041479	0.165269
50%	1.000000	0.106323	0.294195	0.020309	0.081125	0.346462
75%	2.000000	0.180921	0.391065	0.058467	0.147231	0.640236
max	10.000000	1.000000	1.000000	1.307692	0.800313	0.826000

```
# Set academic style
sns.set_theme(style="whitegrid")
fig, axes = plt.subplots(3, 2, figsize=(14, 16), dpi=200)
axes = axes.flatten()

features = [
    "PovertyRate",
```

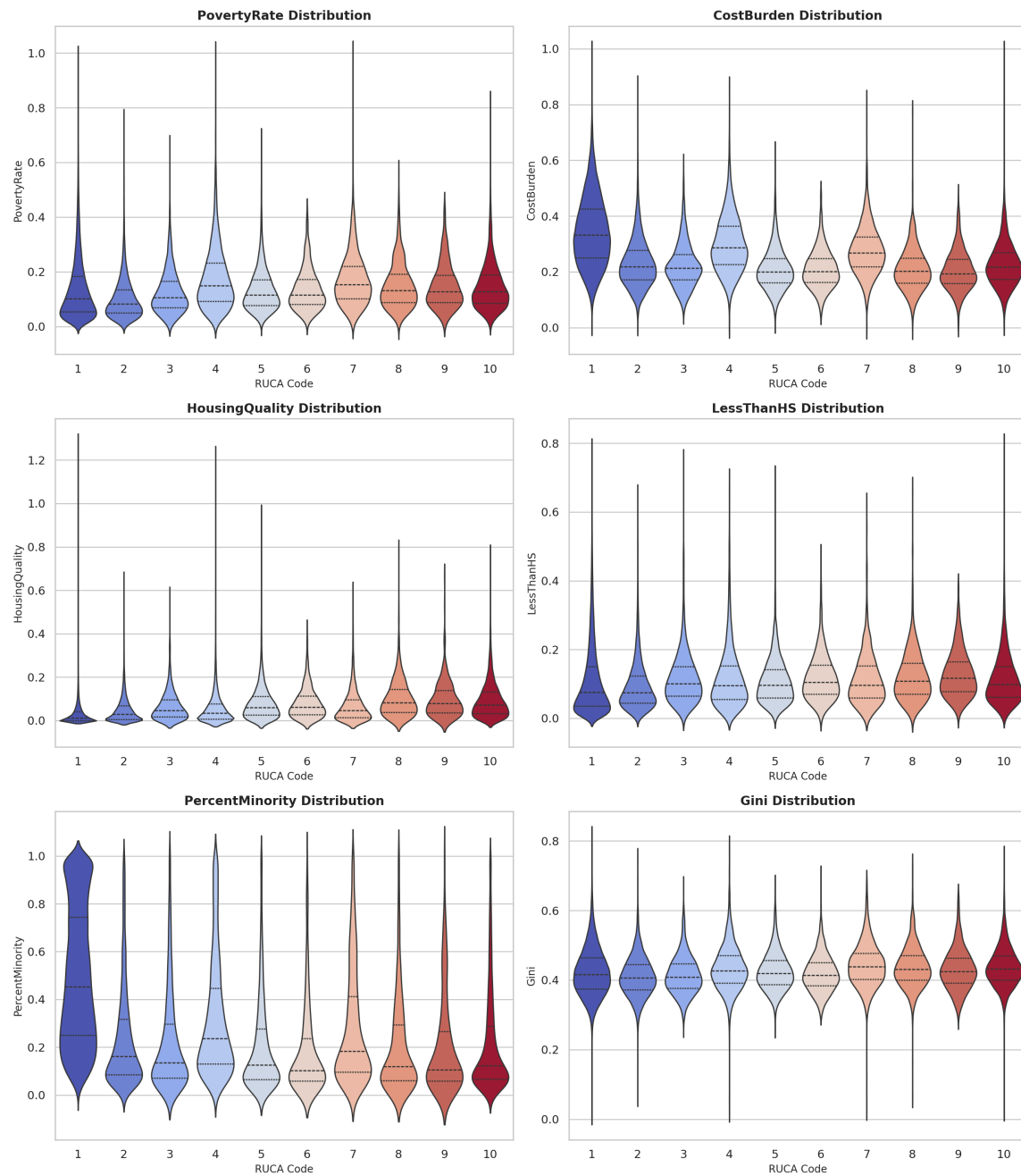
```

    "CostBurden",
    "HousingQuality",
    "LessThanHS",
    "PercentMinority",
    "Gini",
]
colors = sns.color_palette("muted", len(features))

for idx, (feature, ax) in enumerate(zip(features, axes)):
    sns.violinplot(
        x="RUCA",
        y=feature,
        data=df,
        ax=ax,
        palette="coolwarm",
        hue="RUCA",
        legend=False,
        inner="quartile",
        linewidth=1.2,
    )
    ax.set_title(f"{feature} Distribution", fontsize=12, fontweight="bold")
    ax.set_xlabel("RUCA Code", fontsize=10)
    ax.set_ylabel(feature, fontsize=10)

plt.tight_layout()
plt.savefig("ruca_violin_panel.png", bbox_inches="tight")
plt.show()

```



3.2 Pearson Correlation Matrix

```
# Select numerical columns of interest
numerical_cols = [
    "RUCA",
    "PovertyRate",
    "CostBurden",
    "HousingQuality",
    "LessThanHS",
    "PercentMinority",
    "Gini",
]
corr_matrix = DATA[numerical_cols].dropna(axis=0).corr(method="pearson")

# Plot setup
fig, ax = plt.subplots(figsize=(8, 7), dpi=150)
im = ax.imshow(corr_matrix, cmap="coolwarm", vmin=-1, vmax=1)

# Add colorbar
cbar = ax.figure.colorbar(im, ax=ax, shrink=0.75, pad=0.05)
cbar.ax.tick_params(labelsize=10)
cbar.outline.set_visible(False)

# Show ticks and labels
ax.set_xticks(np.arange(len(numerical_cols)))
ax.set_yticks(np.arange(len(numerical_cols)))
ax.set_xticklabels(
    numerical_cols, rotation=45, ha="right", rotation_mode="anchor", fontsize=10
)
ax.set_yticklabels(numerical_cols, fontsize=10)

# Hide the spines
for edge in ["top", "right", "bottom", "left"]:
    ax.spines[edge].set_visible(False)

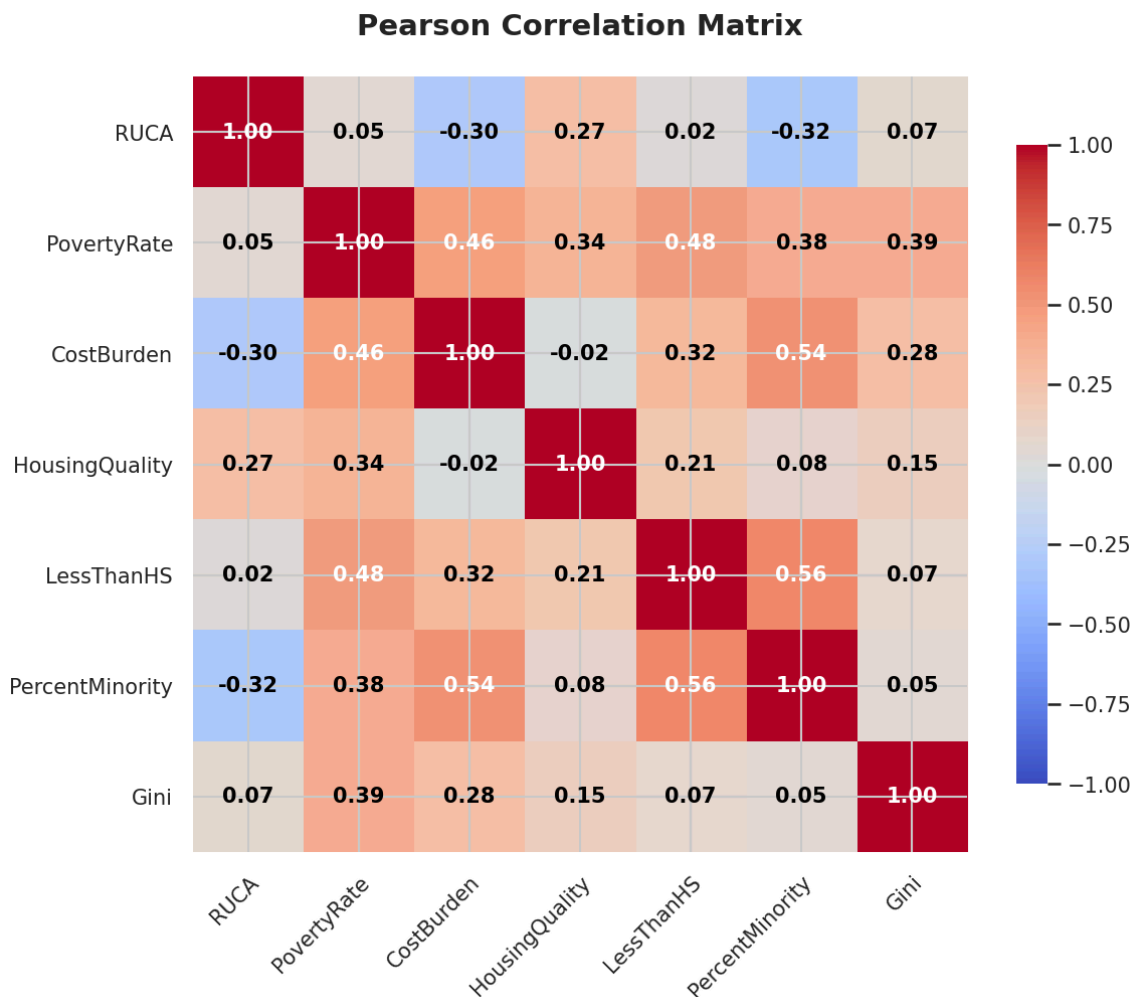
# Loop over data dimensions and create text annotations
for i in range(len(numerical_cols)):
    for j in range(len(numerical_cols)):
        val = corr_matrix.iloc[i, j]
        # Choose text color based on cell color (contrast)
        color = "white" if abs(val) > 0.4 else "black"
        text = ax.text(
            j,
            i,
```

```

        f"{val:.2f}",
        ha="center",
        va="center",
        color=color,
        fontsize=10,
        weight="bold",
    )

# Title and layout
ax.set_title("Pearson Correlation Matrix", fontsize=14, weight="bold", pad=20)
fig.tight_layout()
plt.show()

```



- The minority percentage and cost burden percentage of a census tract decreases as census tracts become more urban whereas poor housing quality go's up.

- LessThanHS has decent positive correlations (0.3 - 0.5) with PovertyRate and PercentMinority

3.3 Variable Inflation Factor

```
# Keep only numeric columns (excluding the target RUCA variable)
vif_features = [
    "PovertyRate",
    "CostBurden",
    "HousingQuality",
    "LessThanHS",
    "PercentMinority",
    "Gini",
]

df_vif = df[vif_features].astype(float)

vif_data = pd.DataFrame()
vif_data["feature"] = df_vif.columns

vif_data["VIF"] = [
    variance_inflation_factor(df_vif.values, i) for i in
    range(len(df_vif.columns))
]
vif_data
```

	feature	VIF
0	PovertyRate	4.343494
1	CostBurden	12.310752
2	HousingQuality	1.680772
3	LessThanHS	3.834566
4	PercentMinority	5.660608
5	Gini	8.526785

- VIF Interpretation
 - $VIF \approx 1$: No correlation with other predictors
 - $1 < VIF \leq 5$: Mild to moderate correlation (usually fine)
 - $VIF > 10$: Strong multicollinearity -> take corrective steps

CostBurden and Percent Minority are over 5 but less than 6, so I don't think they'll cause significant multicollinearity issues.

At 8.5 the Gini coefficient is close to being problematic. Since it's the main way the ACS measures income inequality, so unless it causes problems down the road I guess it's fine ^\(^)/-

3.4 Multinomial Logistic Regression

```
X = df[
    [
        "PovertyRate",
        "CostBurden",
        "HousingQuality",
        "LessThanHS",
        "PercentMinority",
        "Gini",
    ]
].astype(float)
X = sm.add_constant(X)
y = df["RUCA"].astype(int)

# Fit the model
model = sm.MNLogit(y, X)
result = model.fit(method="bfgs", maxiter=1000)

# Display the results
print(result.summary())
```

Optimization terminated successfully.

Current function value: 0.931436

Iterations: 375

Function evaluations: 377

Gradient evaluations: 377

MNLogit Regression Results

Dep. Variable:	RUCA	No. Observations:	81802
Model:	MNLogit	Df Residuals:	81739
Method:	MLE	Df Model:	54
Date:	Sun, 07 Jun 2026	Pseudo R-squ.:	0.2071
Time:	13:39:49	Log-Likelihood:	-76193.
converged:	True	LL-Null:	-96090.
Covariance Type:	nonrobust	LLR p-value:	0.000

	RUCA=2	coef	std err	z	P> z	[0.025	0.975]

const		0.7639	0.091	8.381	0.000	0.585	0.943
PovertyRate		-0.6955	0.220	-3.164	0.002	-1.126	-0.265
CostBurden		-8.1085	0.171	-47.294	0.000	-8.445	-7.772
HousingQuality		6.3021	0.232	27.109	0.000	5.846	6.758

LessThanHS	9.4826	0.214	44.409	0.000	9.064	9.901
PercentMinority	-5.0494	0.081	-62.492	0.000	-5.208	-4.891
Gini	0.6074	0.231	2.632	0.008	0.155	1.060

RUCA=3	coef	std err	z	P> z	[0.025	0.975]

const	-1.3342	0.224	-5.956	0.000	-1.773	-0.895
PovertyRate	2.3709	0.472	5.024	0.000	1.446	3.296
CostBurden	-10.0531	0.426	-23.574	0.000	-10.889	-9.217
HousingQuality	8.2449	0.433	19.042	0.000	7.396	9.094
LessThanHS	13.2341	0.437	30.263	0.000	12.377	14.091
PercentMinority	-6.8611	0.193	-35.506	0.000	-7.240	-6.482
Gini	0.8517	0.563	1.512	0.130	-0.252	1.955

RUCA=4	coef	std err	z	P> z	[0.025	0.975]

const	-1.6676	0.109	-15.243	0.000	-1.882	-1.453
PovertyRate	5.4197	0.177	30.537	0.000	5.072	5.768
CostBurden	-3.9845	0.179	-22.309	0.000	-4.335	-3.634
HousingQuality	4.9088	0.239	20.550	0.000	4.441	5.377
LessThanHS	6.5278	0.224	29.102	0.000	6.088	6.967
PercentMinority	-4.5614	0.089	-51.123	0.000	-4.736	-4.387
Gini	1.1015	0.266	4.145	0.000	0.581	1.622

RUCA=5	coef	std err	z	P> z	[0.025	0.975]

const	-1.4349	0.164	-8.726	0.000	-1.757	-1.113
PovertyRate	4.2239	0.340	12.406	0.000	3.557	4.891
CostBurden	-13.3378	0.333	-40.054	0.000	-13.991	-12.685
HousingQuality	9.7721	0.299	32.669	0.000	9.186	10.358
LessThanHS	11.3313	0.368	30.781	0.000	10.610	12.053
PercentMinority	-6.9921	0.146	-47.879	0.000	-7.278	-6.706
Gini	4.1201	0.405	10.172	0.000	3.326	4.914

RUCA=6	coef	std err	z	P> z	[0.025	0.975]

const	-1.9850	0.286	-6.939	0.000	-2.546	-1.424
PovertyRate	4.5077	0.558	8.073	0.000	3.413	5.602
CostBurden	-12.5656	0.572	-21.986	0.000	-13.686	-11.445
HousingQuality	9.6827	0.489	19.793	0.000	8.724	10.642
LessThanHS	13.7956	0.554	24.892	0.000	12.709	14.882
PercentMinority	-8.1782	0.256	-31.979	0.000	-8.679	-7.677
Gini	2.0686	0.710	2.913	0.004	0.677	3.460

	RUCA=7	coef	std err	z	P> z	[0.025	0.975]
const		-3.5833	0.184	-19.433	0.000	-3.945	-3.222
PovertyRate		5.5806	0.306	18.224	0.000	4.980	6.181
CostBurden		-6.2996	0.317	-19.884	0.000	-6.921	-5.679
HousingQuality		6.8768	0.349	19.714	0.000	6.193	7.560
LessThanHS		7.8038	0.396	19.699	0.000	7.027	8.580
PercentMinority		-5.5173	0.155	-35.610	0.000	-5.821	-5.214
Gini		4.3665	0.442	9.884	0.000	3.501	5.232
	RUCA=8	coef	std err	z	P> z	[0.025	0.975]
const		-3.8594	0.295	-13.087	0.000	-4.437	-3.281
PovertyRate		5.0861	0.562	9.053	0.000	3.985	6.187
CostBurden		-14.1110	0.590	-23.910	0.000	-15.268	-12.954
HousingQuality		11.2457	0.431	26.065	0.000	10.400	12.091
LessThanHS		12.7613	0.604	21.123	0.000	11.577	13.945
PercentMinority		-7.8149	0.251	-31.090	0.000	-8.308	-7.322
Gini		6.5252	0.706	9.248	0.000	5.142	7.908
	RUCA=9	coef	std err	z	P> z	[0.025	0.975]
const		-3.5497	0.392	-9.046	0.000	-4.319	-2.781
PovertyRate		5.2732	0.720	7.324	0.000	3.862	6.684
CostBurden		-15.0318	0.801	-18.757	0.000	-16.603	-13.461
HousingQuality		10.4987	0.594	17.660	0.000	9.334	11.664
LessThanHS		14.1473	0.726	19.473	0.000	12.723	15.571
PercentMinority		-7.8935	0.327	-24.111	0.000	-8.535	-7.252
Gini		4.6274	0.953	4.855	0.000	2.759	6.495
	RUCA=10	coef	std err	z	P> z	[0.025	0.975]
const		-1.7874	0.110	-16.211	0.000	-2.003	-1.571
PovertyRate		4.0689	0.222	18.322	0.000	3.634	4.504
CostBurden		-11.1487	0.212	-52.605	0.000	-11.564	-10.733
HousingQuality		10.8718	0.214	50.712	0.000	10.452	11.292
LessThanHS		11.8330	0.249	47.512	0.000	11.345	12.321
PercentMinority		-7.3449	0.100	-73.477	0.000	-7.541	-7.149
Gini		6.0515	0.269	22.535	0.000	5.525	6.578

Pretty much all of the coefficients are statistically significant but considering the large number of observations and the very low amount of variation between values, I don't think it really means much beyond being able to say they're statistically significant.

```
# statsmodels MNLogit results provide parameters and conf_int per outcome
params = result.params # DataFrame of shape (num_features, num_classes-1)
conf_int = result.conf_int() # shape: (num_classes-1, num_features, 2)

features_of_interest = [
    "CostBurden",
    "HousingQuality",
    "LessThanHS",
    "PercentMinority",
    "Gini",
]
ruca_classes = list(range(2, 11))

fig, ax = plt.subplots(figsize=(10, 6), dpi=200)

for feature in features_of_interest:
    coef_vals = [params.loc[feature, c - 2] for c in ruca_classes]

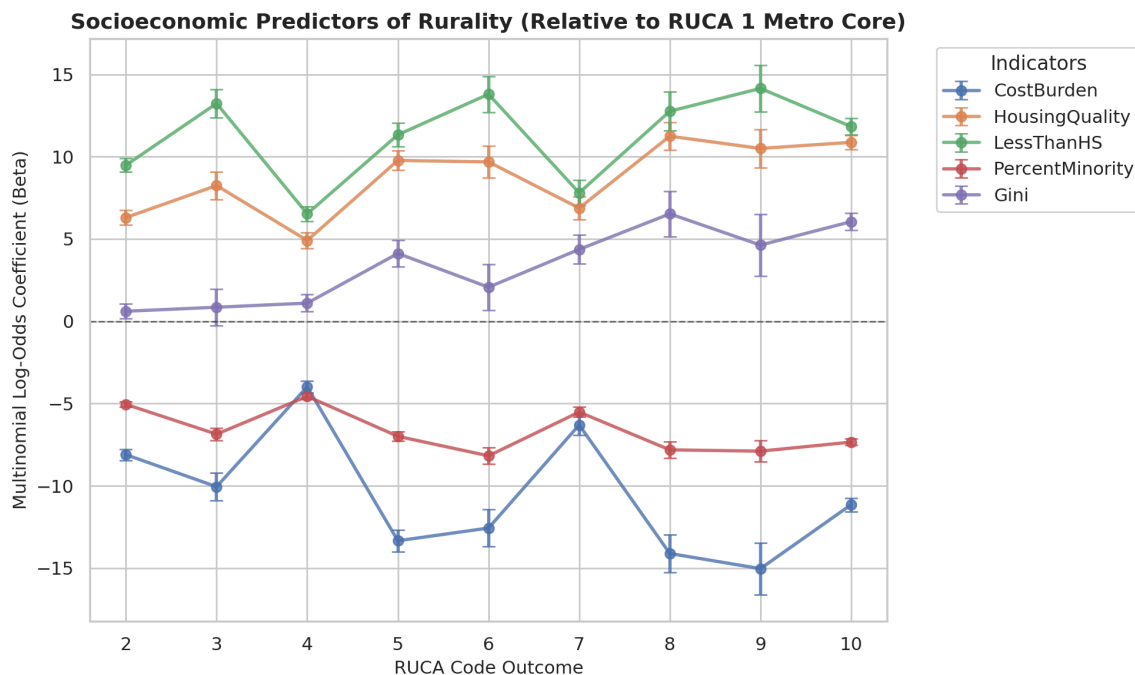
    # Calculate confidence interval ranges (95% CI)
    lower_vals = [
        result.conf_int().loc[(str(c), feature), "lower"] for c in ruca_classes
    ]
    upper_vals = [
        result.conf_int().loc[(str(c), feature), "upper"] for c in ruca_classes
    ]
    yerr = [
        np.array(coef_vals) - np.array(lower_vals),
        np.array(upper_vals) - np.array(coef_vals),
    ]

    ax.errorbar(
        ruca_classes,
        coef_vals,
        yerr=yerr,
        label=feature,
        fmt="-o",
        capsize=4,
        alpha=0.8,
        linewidth=2,
    )
```

```

ax.axhline(0, color="black", linestyle="--", linewidth=1, alpha=0.5)
ax.set_title(
    "Socioeconomic Predictors of Rurality (Relative to RUCA 1 Metro Core)",
    fontsize=13,
    fontweight="bold",
)
ax.set_xlabel("RUCA Code Outcome", fontsize=11)
ax.set_ylabel("Multinomial Log-Odds Coefficient (Beta)", fontsize=11)
ax.set_xticks(ruca_classes)
ax.legend(title="Indicators", bbox_to_anchor=(1.05, 1), loc="upper left")
plt.tight_layout()
plt.savefig("mlr_coefficient_trajectories.png", bbox_inches="tight")
plt.show()

```



Positive multinomial log-odds coefficient: variable increase -> likelihood of class (RUCA code) increases.

```

# Predict probabilities of classification on the training data
probs = result.predict(X)

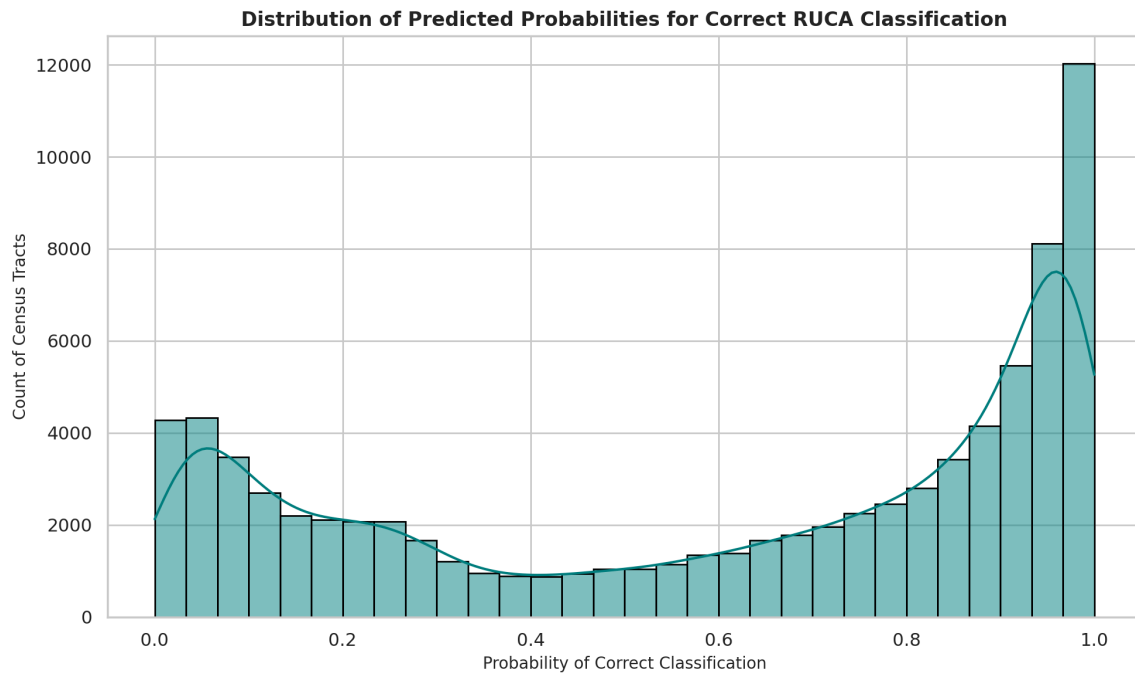
# Extract correct classification probability for each tract
df["RucaProb"] = probs.values[np.arange(len(y)), y - 1]

```

```
# Display summary statistics of the correct classification probabilities
print(
    "Mean probability of correct classification:", round(df["RucaProb"].mean()
* 100, 2)
)
print("Median      probability      of      correct      classification:",
np.median(df["RucaProb"]))
```

```
Mean probability of correct classification: 59.81
Median probability of correct classification: 0.7298630414056722
```

```
plt.figure(figsize=(10, 6), dpi=200)
sns.histplot(df["RucaProb"], bins=30, kde=True, color="teal",
edgecolor="black")
plt.title(
    "Distribution of Predicted Probabilities for Correct RUCA Classification",
    fontsize=12,
    fontweight="bold",
)
plt.xlabel("Probability of Correct Classification", fontsize=10)
plt.ylabel("Count of Census Tracts", fontsize=10)
plt.tight_layout()
plt.show()
```



4 Spatial Analysis